FEAT-ZD-48600 · DE-01 Checking the coding prerequisites ■ · Repository (and Databricks) availability and write access

Data Engineering agent · item task.repo_access · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.repo_access.md · approved by Data engineer sim.data_engineer

task.repo_access: FEAT-ZD-48600

DE-01 Checking the coding prerequisites for the Data Engineering lane of FEAT-ZD-48600. The step checks Databricks access, whether the repository is available, whether a repository has to be created, and write access. Gate: automated checks, the independent judge and sampled human evaluation (record:brief).

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-01 Checking the coding prerequisites	record:brief
Trigger	Task creation (feat.tasks from DES-04)	record:brief; feat.tasks
Inputs	feat.tasks (b0fd4d2), org.engineering_stack (3b03f15)	record:brief
Data Engineering tasks in scope	TSK-01 (STORY-01, `paid_date` in `transform.to_model`) and TSK-02 (STORY-02, new `delays.delays_by_vendor`), both in ap_pipeline	feat.tasks (sections 1, 3)

1. Result

Prerequisite	Status	Evidence	Source
Repository needed	ap_pipeline only. TSK-01 and TSK-02 both name the Data Engineering lane and ap_pipeline	feat.tasks section 1	feat.tasks
Repository available	**Yes.** ap_pipeline is checked out locally. Code is in `src/ap_pipeline/` (`__init__.py`, `dq.py`, `ingest.py`, `pipeline.py`, `schedule.json`, `transform.py`) and tests are in `tests/` (`test_pipeline.py`). `ci.py` and `data/sample_feed.csv` are at the root	Repository file listing; repository memory	tool:repo-access; file:ap_pipeline/CLAUDE.md
Repository creation	**Not needed.** The repository exists. TSK-02 adds `src/ap_pipeline/delays.py` and `tests/test_delays.py` inside it	Tasks add modules to an existing repository	feat.tasks (TSK-02); tool:repo-access
Branches	`main`, `dev` and `feature/story` exist. HEAD is `feature/story`, created from `dev` at `3fcbe1b`. `main`, `dev` and `feature/story` all point to the same commit	`.git/HEAD`, `.git/refs/heads/*`	tool:repo-access; file:ap_pipeline/.git/HEAD; file:ap_pipeline/.git/refs/heads/feature/story; file:ap_pipeline/.git/refs/heads/dev; file:ap_pipeline/.git/refs/heads/main
Write access	**Yes.** The agent identity has write access and the working tree is clean	Spine tool view	tool:repo-access

Prerequisite	Status	Evidence	Source
Databricks access	**Not applicable.** No repository uses Databricks or a database. The ap_pipeline source has no Databricks, Spark, JDBC or database reference, so there is no workspace to check	org.engineering_stack; tool view; code search with no matches	org.engineering_stack; tool:repo-access
Language and CI	Python 3.9 standard library. CI is `/usr/bin/python3 ci.py` with line coverage of at least 90% plus static checks that stand in for SonarQube	Repository memory and stack	org.engineering_stack; file:ap_pipeline/CLAUDE.md; file:ap_pipeline/ci.py
Current version	ap_pipeline `0.4.0`, kept in `src/ap_pipeline/__init__.py`. TSK-01 owns the version step	`src/ap_pipeline/__init__.py`	file:ap_pipeline/src/ap_pipeline/__init__.py; file:ap_pipeline/CLAUDE.md; feat.tasks (section 2)

Outcome: the repository is available and has write access. No repository needs to be created. Databricks is not used, so it is not a prerequisite for TSK-01 or TSK-02. The Data Engineering lane can go on to DE-02 (record:brief; tool:repo-access; org.engineering_stack).

2. Branch and commit conventions for the lane

Item	Value	Source
Branches	`main` (released), `dev` (integration), `feature/<story>`	file:ap_pipeline/CLAUDE.md; feat.tasks (section 2)
Working branch	`feature/story`, created from `dev`	tool:repo-access
Commit message	Starts with the story ID (`STORY-01` / `STORY-02`). The task ID may follow it	feat.tasks (section 2); org.engineering_stack; record:memory
Signatures	Keep the existing public signatures and add new functions for new behaviour	file:ap_pipeline/CLAUDE.md
Push and merge	Only the spine pushes and merges. This step ran no git commands that change state	record:brief

3. Notes for later Data Engineering steps

ID	Note	Source
N-1	`src/ap_pipeline/schedule.json` sets a daily run at `02:00 Asia/Kolkata`. TSK-01 lists `schedule.json` under "Must not change". This matters for DE-13 scheduling	file:ap_pipeline/src/ap_pipeline/schedule.json; feat.tasks (TSK-01)
N-2	`ci.py` imports the third-party `coverage` package to measure coverage. This step could not run shell commands, so it could not confirm that `coverage` is installed for `/usr/bin/python3`. The first CI run will confirm it	file:ap_pipeline/ci.py
N-3	TSK-01 does not depend on any code. Feed v3 in production (PQ-6) gates release, not coding. Build on synthetic v3 rows only	feat.tasks (TSK-01)

4. Validation check

Check	Result	Source
Every repository in the Data Engineering tasks is available with write access	Pass (ap_pipeline)	feat.tasks; tool:repo-access

Check	Result	Source
Databricks checked	Not applicable: no repository uses it	org.engineering_stack; tool:repo-access
No production data, secrets or personal data read or written	Pass	record:brief
Blocking questions	None	—

5. Open questions (non-blocking)

ID	Question	Interim answer	Owner (proposed)
RAQ-1	Is the `coverage` package installed for `/usr/bin/python3`, which `ci.py` needs? It was not checked because DE-01 allows no shell commands	Assume yes. The first CI run (DE-09 / DE-10) confirms it	Data engineer

[Tool result] [ap_pipeline] Branch feature/story at 3fcbe1b; write access confirmed

[Tool result] [ap_pipeline] Branch feature/story at 3fcbe1b; write access confirmed